Can ChatGPT Generate Realistic Synthetic System Requirement Specifications? A Case Study

Anonymous Author(s)

Abstract

System requirement specifications (SyRSs) are central, natural language (NL) artifacts in SE and requirements engineering (RE). Having access to them is highly valuable for SE research. However, access to real data is scarce due to proprietary restrictions, confidentiality concerns, and unavailability. Generating methods for synthetic SyRSs (SSyRSs) can address this problem. Black-box Large Language Models (LLMs) such as ChatGPT offer compelling generation means, due to their strong NL processing capabilities and easy access. However, LLMs suffer from hallucinations and overconfidence, which pose major challenges, and their feasibility for generating synthetic RE artifacts remains unexplored. This paper presents the methodology and results of an exploratory study that examined ChatGPT's capabilities in generating realistic SSyRSs using prompt engineering techniques, self-assessment strategies, and iterative prompt refinements. To this end, we developed SSyRS-Gen—a SSyRS generation and assessment process—and applied it in a case study using GPT-4o. We generated 300 SSyRSs across ten industry domains in total. A questionnaire study (n=87) assessed how realistic a SSyRS sample is perceived by experts. Both ChatGPT (89% on average) and over half of the experts (61%) rated them as realistic. However, more critical analyses of the data revealed significant weaknesses, such as contradictory statements within the SSyRS. It is apparent that even a systematic approach with ChatGPT falls short in producing truly realistic data in expert contexts. Our findings highlight risks in relying on black-box LLMs for complex NL documents and caution against their uncritical use.

Keywords

Requirements Engineering, System Requirement Specifications, Requirement Generation, Synthetic Requirements, Prompt Engineering, AI4RE

ACM Reference Format:

Anonymous Author(s). 2025. Can ChatGPT Generate Realistic Synthetic System Requirement Specifications? A Case Study. In . ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

The System Requirements Specification (SyRS) is a central artifact of Requirements Engineering (RE). According to ISO/IEC/IEEE 29148:2018 [16], it is a structured collection of the functional and non-functional requirements, design constraints, and other related information for the system, its operational environments, and external interfaces. They are mostly documented in natural language (NL), as it is more accessible to all stakeholders than formal languages [18, 22]. Having access to SyRS data is of high value for many research activities in SE research, such as developing methods and tools for elicitation, and validation of requirements [24], automated testing and test case generation [13], model-based software engineering [26], and tool benchmarking and evaluation [11]. However, access to SyRS data is minimal, as acquiring it is often infeasible due to proprietary restrictions, confidentiality concerns, and unavailability [10, 11]. One way to alleviate this is to generate realistic SyRS models. We define a *synthetic system requirement specification* (SSyRS) as an NL artifact that models a SyRS.

To generate SSyRSs, Large Language Models (LLMs) offer compelling opportunities, due to their natural language processing (NLP) capabilities [3, 14]. Black-box LLMs such as ChatGPT are particularly attractive. They offer fast and easy access to generation capabilities for NL artifacts, as they do not require any initial training on the part of the user and are available via directly accessible APIs [31]. However, Mehraj et al. [24] point out that research in the area of LLMs in RE primarily focuses on identifying and analyzing requirements. Hence, the feasibility of LLMs for generating synthetic requirement artifacts remains underexplored. Although LLMs seem attractive for this use case, they come with major challenges that severely limit their usability for generating NL artifacts. *Hallucination* is the phenomenon that LLMs unknowingly generate data that is nonsensical, unfaithful to the input source, or unverifiable [15]. *Overconfidence* describes the tendency that LLMs present information with high certainty, regardless of whether the content is factually correct or verifiable [36]. Both phenomena interplay dangerously: First, an LLM generates incorrect data by hallucinating. Then, it presents the incorrect data in NL in an overly confident manner, which easily convinces the user of the data's correctness [15, 36]. Lastly, generating NL artifacts in expert domains, which is the case for SSyRSs, reveals a particular challenge. LLMs are trained on internet data [25], but expert domain knowledge is shared far less over the internet than common knowledge. This results in significantly smaller—and possibly incorrect—training datasets. This challenge increases the risk of hallucinations, as LLMs would rather generate incorrect data than admit they do not know something [6]. While there exist mitigation strategies for hallucination, they rely on access to internal model parameters, which black-box LLMs do not provide [15]. As a result, alternative methods, such as output verification with external knowledge, auxiliary fact-checking models, or structured prompt engineering, must be employed when using black-box LLM [15, 21, 23].

This paper examines whether ChatGPT is a feasible tool to generate SSyRSs, which can be used in SE research. To this end, we first need to understand whether and to what extent SSyRSs can be

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference'17, Washington, DC, USA

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

PREPRINT

used as substitutes for real SyRSs in SE research. We therefore have to define quality properties and corresponding metrics for SSyRSs that allow conclusions to be drawn about their model properties, such as how realistic a SSyRS is compared to a real SyRSs. This is concluded in the first research question:

RQ1: What quality properties describe the suitability of SSyRSs to be used in SE research as substitutes for real SyRSs?

Our study aims not only to generate SSyRSs but also to evaluate their quality. To this end, we would like to investigate whether ChatGPT can replace experts in evaluating SSyRSs. This summarizes our second research question:

RQ2: How can the quality properties of SSyRSs be measured without domain expert involvement?

Next, as described above, we must leverage prompt engineering techniques to mitigate hallucination and improve the quality of SSyRS. The question remains whether these methods help improve their quality. To this end, targeted, iterative prompt refinements shall be investigated:

RQ3: Can targeted, iterative prompt refinements help to improve the quality properties of SSyRS verifiably?

Finally, if the iterative prompt improvement approach can reach a desired quality level, we want to examine whether human expert assessments of the SSyRSs would reflect these quality assessments:

RQ4: Are assessments of SSyRSs made by human experts in line with ChatGPT-based quality assessments?

Contributions: We selected ChatGPT to generate SSyRSs for two reasons: It is the most prominent black-box LLM, with more than 5.2 billion monthly visits at the time of writing [8], and it offers quick and easy access. It is therefore tempting to use it for generating SyRS-like artifacts in a productive environment. Thus, our results will either support the usage of ChatGPT for generating SyRS-like artifacts or advise against its use. In either case, our study serves as exploratory groundwork and is, to our knowledge, the first of its kind in using LLMs for generating synthetic requirement artifacts.

The paper is structured as such: Section 2 presents our research design. Section 3 introduces SSyRS-Gen, the SSyRS generation process we developed. Section 4 presents a case study by implementing and executing SSyRS-Gen. Section 5 describes expert evaluations of our SSyRSs. The discussion follows in section 6. Section 7 briefly discusses related work. Section 8 concludes the paper.

2 Research Design

Design science research (DSR) is a widely applied and accepted means in information systems research [28]. As multiple publications demonstrated its applicability in SE research [9, 35], our study follows DSR. Next, we will briefly describe how we implemented the six steps defined by DSR.

1—Identify Problem and Motivate: As presented in section 1, having access to SyRSs is a crucial limitation for many research efforts in SE research. Being able to generate synthetic yet realistic SyRSs would, thus, have a huge beneficial impact on SE research.

2—Define Objectives of a Solution: We conducted a focused literature review and discovered compelling use cases of how black-box LLMs can be utilized to generate, reason about, evaluate, and refine synthetic SyRSs [4, 12, 29]. Moreover, we identified the reliability and truthfulness issues introduced in section 1 [14, 15, 36], as well as mitigation strategies in prompting techniques, prompt patterns, and LLM self-assessment strategies [15, 21, 23]. Based on these insights, we identified three qualities that SSyRSs should possess for effective use in SE research:

- **Q1: Realistic** – *A SSyRS must plausibly represent a real-world SyRS.* This is the core quality of SSyRSs to be optimized. The value of using SSyRSs in SE research depends mainly on how well they model real SyRS. To measure the extent to which an SSyRS realistically resembles a real SyRS, we have defined the qualitative *"Degree of Realism"* (DoR) metric.
- **Q2: Comparable** – *SSyRSs must be consistent in their structure and comprehensible to compare SSyRSs.* To enable objective evaluation, SSyRSs must comply with a predefined content structure. To measure the extent to which an SSyRS complies with the expected structure, we defined the *"Completeness"* metric.
- **Q3: Diverse** – *Multiple SSyRSs should represent closely related, yet distinct variants of each other.* Since we want to create a wide range of SSyRSs, they may have minor and significant differences in their semantics, but they must not be redundant. To measure the degree of semantic equivalence between two SSyRSs, we use the *"Semantic Similarity"* metric [5].

In addition, we defined three requirements for the generation process. They reflect the specific objectives and challenges of our study:

- **R1: No External Documentation** – *SSyRSs must be generated and evaluated without accessing external documentation such as real SyRSs.* This requirement acknowledges the scarcity of publicly accessible and evaluated SyRSs. Accordingly, the generation process must be able to generate SSyRSs and evaluate their quality without relying on any external documentation.
- **R2: No Domain Expert Involvement** – *SSyRSs must be generated and evaluated without involving domain experts.* This requirement immediately follows from RQ2.
- **R3: Domain Independent** – *SSyRSs should not be limited to a specific industry domain.* SyRSs are domain-specific. Thus, we consider the industry domain of an SSyRS as a unique defining factor. Enabling the generation of SSyRSs for a broad range of industry domains allows for their use in more diverse settings.

3—Design and Development: The qualities and requirements identified in the previous step, along with the prompting techniques we identified, formed the basis for developing the SSyRS generation process (SSyRS-Gen), in which the prompts for generating and evaluating SSyRSs are iteratively refined.

4—Demonstration: We demonstrate the SSyRS-Gen through a case study. To this end, we defined ten industry domains. For each domain, 10 iterations were performed based on refined generation and self-assessment prompts. In each iteration, three SSyRSs were generated per industry domain. This resulted in a total of

PREPRINT

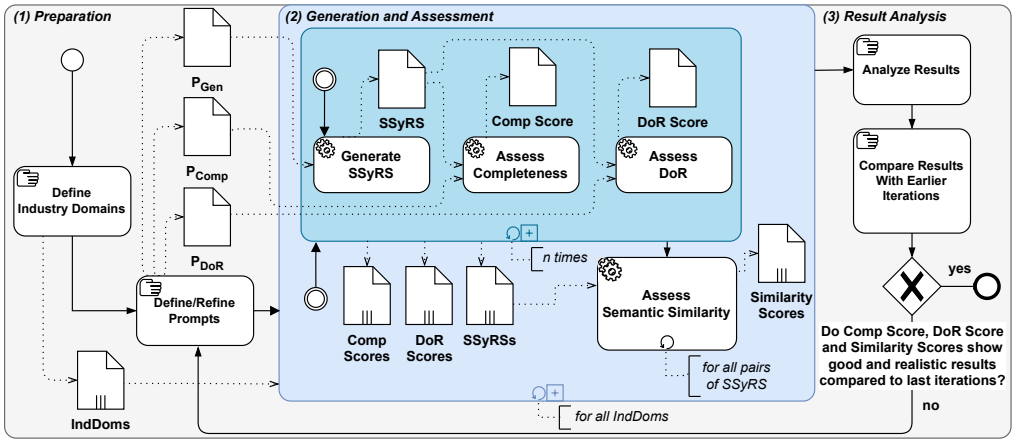

Figure 1: BPMN model of SSyRS-Gen. Colors indicate loops. Italic comments describe loop conditions.

300 SSyRSs, all of which were evaluated according to the qualities Q1-Q3.

5—Evaluation: To evaluate the SSyRSs artifacts and give an answer to RQ4, we conducted an questionnaire study. We aimed to determine whether experts consider SSyRSs to be realistic or artificial. To this end, we selected a sample of one SSyRS per industry domain. We designed a questionnaire to gain insights into how realistically the participants perceive these SSyRSs. The questionnaire study was conducted with a total of 87 participants.

6—Communication: The communication is done by scientific publications, such as this paper. We also plan on future publications for SSyRS-Gen and extended case studies.

3 The SSyRS-Gen Process

The core of SSyRS-Gen is iterative prompt refinements based on the analysis of earlier process executions. It is depicted as a BPMN model in Figure 1. Manual steps are performed by prompt engineers (PEs), automated steps by ChatGPT or other scripts. Overall, SSyRS-Gen is executed in multiple iterations and can be divided into three distinct phases. The phases can be identified in the BPMN model in figure 1 by the left, middle, and right parts of the model.

The process begins with the *(1) Preparation Phase*. Here, all data needed for the generation and assessment of SSyRSs is prepared. To meet R3, in the first iteration, PEs define a set of industry domains (*IndDoms*) for which SSyRS-Gen must generate SSyRSs. Then, to meet R1 and R2, the PEs develop an initial set of three distinct prompts: a generation prompt for SSyRSs (P_{Gen}), one self-assessment prompt to measure the completeness of SSyRSs (P_{Comp}) for Q2, and one self-assessment prompt to measure the DoR of SSyRSs (P_{DoR}) for Q1. Defining a prompt to measure semantic similarity for Q3 is not required. SSyRS-Gen relies on self-assessment strategies to assess Q1 and Q2 in order to fulfill R1 and R2. Such strategies come with a set of disadvantages regarding the accuracy and precision of the estimations [33]. These can be mitigated when calculating semantic similarity while still fulfilling R1 and R2 by leveraging more reliable solutions, such as Sentence Transfomers (SBERT) [30], which detect semantic similarities with high accuracy.

After defining the prompts, the *(2) Generation and Assessment Phase* is processed. Here, SSyRSs are being generated and their quality properties Q1-Q3 are assessed. This phase is iteratively executed for all IndDoms (blue area). Each iteration further executes a second, inner iteration (cyan area). The inner iteration is executed n times, where n is the number of desired SSyRSs to be generated for one IndDom. This design decision aims to improve Q2 of the SSyRSs. By allowing multiple SSyRSs to be generated for the same IndDom, SSyRSs can easier be compared with each other. In the inner iteration, ChatGPT first receives P_{Gen} as input to generate a SSyRS. It then receives the SSyRS and P_{Comp} to assess the completeness (*Comp score*), and the SSyRS and P_{DoR}, to assess its DoR (*DoR score*). This is repeated for the desired amount of n SSyRSs per IndDom, eventually resulting in n SSyRSs for one IndDom. Last, for all pairs of SSyRS for one IndDom, the semantic similarity scores (*similarity scores*) between each pair are calculated, resulting in $\frac{n \times (n-1)}{2}$ similarity scores. This process is repeated for all IndDoms.

The *(3) Analysis Phase* starts after the second phase was executed for all IndDoms. In this phase, the PEs manually analyze and compare all SSyRSs and their assessments with the results from earlier iterations. The main objective is to decide whether the quality assessments show both good and realistic results compared to previous iterations. If so, the process is terminated. If not, another execution of SSyRS-Gen is performed, by first refining the prompts based on the insights from the analysis, then generating and assessing SSyRSs, and lastly analyzing the results again. There are three reasons for this subjective termination condition: First, LLMs are probabilistic by nature [2]. Since prompt refinements cannot guarantee improvements in the generated data, results of an iteration must be considered in conjunction with previous results to understand their overall performance. Second, the reliability of LLM judgments is a huge concern [33]. We were uncertain whether the self-assessment prompts would lead to reliable judgments about the quality of the artifacts. Third, we had no comparison data for

our metrics. Defining thresholds for the scores as termination conditions was, thus, infeasible. Instead, we had to consider and interpret these results in their overall contexts.

4 Generating SSyRSs with SSyRS-Gen

We conducted a case study using SSyRS-Gen to generate three SSyRSs for ten IndDoms each. We agreed on these numbers as they ensured Q2 and Q3, but kept the human effort in result analysis manageable. Each iteration applied a structured prompting approach, drawing on techniques from White et al. [38] and Schulhoff et al. [34]. Due to space reasons, all prompts and detailed prompt changes are documented and provided in our artifacts [1]; here, we describe the main prompt characteristics and decisions we made.

4.1 Preparation

We defined the following set of ten IndDoms for SSyRS-Gen: 1) E-Commerce *(e-com)*; 2) Education *(edu)*; 3) Finance *(fin)*; 4) Government and Public Services *(gov)*; 5) Healthcare *(heal)*; 6) Logistics *(logi)*; 7) Manufacturing *(manu)*; 8) Media-Entertainment *(media)*; 9) Retail and Supply Chain *(ret)*; 10) Telecommunications *(tele)*. They were generated by a GPT-4o model and manually approved as suitable candidates for our use case. To this end, we designed a prompt that contained the generation instruction and the restriction to not generate any niche domains for reasons of comprehensibility.

4.1.1 Generation prompt: We first defined a *SSyRS-template*. It followed a simplified structure of SyRSs as defined in the ISO/IEC/IEEE 29418 standard [16]. We decided on a simplified structure to keep the human effort for analysis and quality assessments manageable. The template defined four description categories, which reflect the overall common structure of SyRSs: *System Overview; Functional Requirements; Non-functional Requirements; Constraints*. Each category further defined multiple properties, e.g, for System Overview, the property *System Purpose* was defined as a "high-level description of the system's main objectives and goals". Lastly, we defined authentication and data protection requirements for the functional requirements category, as we wanted to observe how well ChatGPT handles specific rather than more general data. The SSyRS-template was modified once in the fifth iteration by adding more properties.

P_{Gen} incorporated the template prompt pattern using the SSyRS-template and followed a zero-shot prompting strategy, i.e., no SyRS examples were provided in the prompt. We decided this for two reasons: to fulfill R1 in the first iteration, as example SyRSs are external documentation, and to not negatively impact the quality of SSyRSs by providing bad example SSyRSs, as our quality assessments were limited to subjective evaluations and unreliable self-assessed DoR scores. P_{Gen} further implemented chain-of-thought, as LLMs are better in solving multiple smaller problems than one complex one [34, 37]. The persona pattern was added to P_{Gen} in the form of an *experienced requirements engineer and business analyst* in the third iteration onward, as it hugely impacts the quality of generated data. Due to space reasons, we cannot share a whole example. Moreover, P_{Gen} showed huge variations across the iterations due to the prompt refinements. Thus, we share the main instruction that all variations had in common. It is depicted below. Note that we used the generic term "scenario" instead of "SSyRS". to reduce possible hallucinations. Since GPT models are trained on internet data, we were concerned that incorrect or vague uses of the term "SyRS" could encourage hallucinations; a risk that might be mitigated by using more generic instructions.

```
Generate a scenario based on the following template for the sector
<IndDom> and ensure that the scenario is distinct to previously
created ones. Template: <SSyRS-template>
```

4.1.2 Self-assessment prompts: P_{Comp} was composed of a straightforward, simple instruction to compare the content of SSyRSs with the SSyRS-template and return false if any category or property as defined in the SSyRS-template was missing. Due to perfectly accurate assessments from the beginning, we just slightly modified it in the first two iterations to enhance the result structure. For P_{DoR}, the first two iterations were simple instructions on how to evaluate a SSyRS, evaluated as a boolean. No prompt patterns were utilized in these iterations. From the third iteration onward, we changed the assessment to a numeric interval $[0, 1]$ for finer assessments, where a DoR score of 1 is interpreted as "perfectly realistic". Furthermore, the persona and chain-of-thought patterns were included in the prompt. In addition to the quantitative assessment, P_{DoR} defined specific evaluation criteria that ChatGPT should use for the assessment. For each finding that resulted in a worsening of the DoR score, ChatGPT had to add a detailed justification. The assessments, thus, provided not only a quantitative measure but also qualitative justifications for the assessment. These justifications were crucial for our prompt refinements, as they enabled us to directly address identified weaknesses in the refinements of the prompts. Due to space reasons, we do not share a whole example of P_{DoR}. Instead, we show an excerpt of the evaluation criteria incorporated in P_{DoR} only. It is depicted below and represents the state of P_{DoR} in the seventh iteration:

```
- Assess whether the created scenario is realistic for the given
  domain of <IndDom>.
- Provide a decimal score between 0 and 1, where 1 indicates the
  scenario is entirely realistic, and 0 indicates it is entirely
  unrealistic.
- If you identify any points as unrealistic, list only those
  points along with a justification for each. Indicate how much
  you deduct from the score for each unrealistic point. Always end
  with the final combined: 'Final Realism Score: <Final Score>'
```

4.2 Generation and Assessment

We used ChatGPT-4o for prompting from January 13, 2025, to January 25, 2025. We opted for ChatGPT over the OpenAI API due to reduced costs, as our tasks involved the generation of many tokens, and for the 4o models as they were outperforming other GPT-models at the time of the study. In each iteration, we used one context for SSyRS generation and assessment per IndDom, i.e., ten different contexts per iteration, and 100 different contexts in total. To calculate similarity scores, we used the Sentence Transformers (SBERT) with the pre-trained `all-mpnet-base-v2` model and cosine similarity due to high accuracy across many semantic textual similarity tasks [30]. A similarity score of 1 is interpreted as "semantically equivalent", a score of 0 as "100% disjoint".

PREPRINT

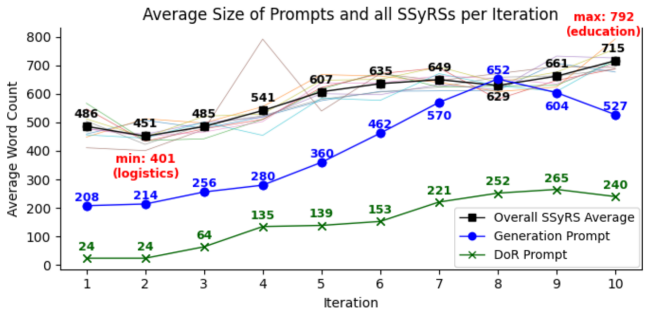

Figure 2: Size growth by number of words of P_{Gen}, P_{DoR}, and average SSyRSs across all industry domains. Light lines show average SSyRSs size per domain for illustrative purposes.

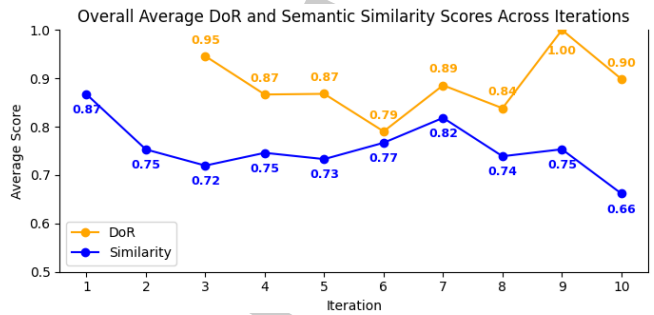

Figure 3: Average Semantic DoR and similarity scores across all industry domains. DoR scores for iterations 1 and 2 are missing as they were assessed as boolean in these iterations.

4.3 Result Analysis

We analyzed the Comp scores manually to identify incomplete SSyRSs and detect errors in the assessments. For the analysis of DoR and similarity scores, we calculated statistical measures such as the mean and median, ranges, and standard deviations of both scores. Then, the analysis was conducted in two steps. First, a cross-IndDom analysis of both values was performed to establish general comparability of the results with the previous iterations and detect domain-specific patterns. To this end, we compared the statistical values with those from previous iterations. Then, we analyzed both scores within the IndDoms to reveal inconsistencies and outliers within an IndDom. For this analysis, we developed Python scripts to visualize trends in their scores and statistical values across current and previous iterations. They enabled comparative evaluations and provided clearer insights into how previous prompt changes affected the results. In addition to quantitative comparisons of the DoR scores, we analyzed the justifications ChatGPT delivered for each weakness it identified in the SSyRSs. We used these justifications to strategically refine P_{Gen} for mitigating the weakness in the next iteration, and P_{DoR} if the justifications were wrong or imprecise. Lastly, we conducted targeted reviews of the SSyRSs with two goals. First, to detect superficial or overly generic language, such as "the system should be reliable," which lacks concrete, domain-specific detail. Second, to give a subjective assessment of whether the SSyRSs of an IndDom and across all IndDoms subjectively appear to be improved compared to previous iterations.

4.4 Results

Our termination condition was met after ten iterations. In the following, the average size and quality measures will be presented. For reasons of space, detailed results for each SSyRS, IndDom, and iteration are not shown. These can be found in the artifacts [1].

4.4.1 Size measures. Figure 2 depicts the size growth in terms of the number of words of P_{Gen}, P_{DoR}, and the average size of SSyRSs over the ten iterations of SSyRS-Gen. P_{Comp} is not depicted as we modified it only slightly in the first two iterations. Its assessments were very accurate from the beginning and always resulted true. Manual verification confirmed the correctness of these assessments. The black line shows the average SSyRS size across all IndDoms, light lines indicate the SSyRS size per IndDom. They are only visualized for illustrative purposes.

In the first iteration, P_{Gen} contained 208, P_{DoR} 24 words. Both prompts grew in size during process execution: P_{Gen} by 213% to the eighth iteration (652 words), P_{DoR} by 1004% to the ninth iteration (265 words). Both prompts decreased from here on to the tenth iteration: P_{Gen} by 19% to 527 words, P_{DoR} by 10% to 540 words. The reasons will be explained later. SSyRSs started with an average size of 486 words across all IndDoms in the first iteration and grew by 47% to 715 words in the tenth iteration. This trend shows that the refinements of P_{Gen} yielded bigger SSyRSs overall. However, they did not grow as much as P_{Gen}. The average total size of SSyRSs across all IndDoms and iterations is 586 words, with a median size of 604 words, indicating that SSyRSs from earlier iterations are smaller than from later ones. This trend is visible in figure 2 as an almost consistent increase from the first to the last iteration. Moreover, the SSyRSs show a high variability across iterations and IndDoms. The average size per IndDom ranged from 401 words in the second iteration for Logistics to 792 words in the tenth iteration for Education.

4.4.2 Quality measures. Figure 3 depicts the average similarity and DoR scores across all IndDoms over ten iterations. DoR scores for the first two iterations are not depicted, as ChatGPT assessed them as boolean in these iterations, with all results being true.

Average similarity: The overall average similarity score ranges from 0.87 in the first, to 0.66 in the last iteration, with an overall average of 0.76 in total. This trend indicates that refinements of P_{Gen} improved the quality of SSyRSs in terms of their semantic similarity. However, this result should be treated with caution, as other possible influencing factors, such as the probabilistic nature of LLMs, cannot be ruled out. Furthermore, this trend is not apparent when excluding the last iteration as an outlier from the analysis; the evolution of the similarity score between the third and seventh iterations shows a clear trend towards a decrease in diversity, i.e., a reduction of the desired quality. At the same time, P_{Gen} grew by 122%. Hence, it can be observed that the size development of P_{Gen}, which reflects more sophisticated generation instructions, is not necessarily related to an improvement in the similarity score.

While no clear trends for similarity scores can be identified across IndDoms, trends can be observed within them. Figure 4 depicts

DRAFT

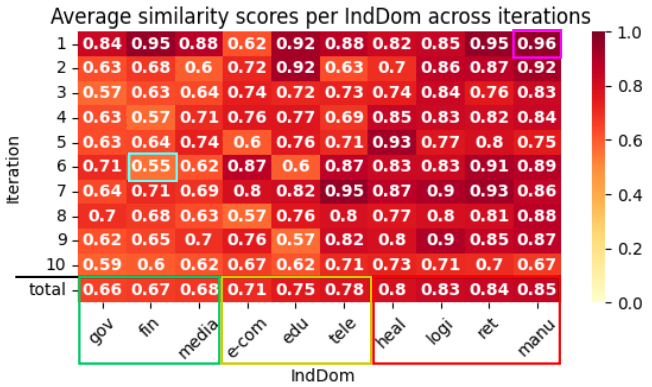

Figure 4: Heatmap of average similarity scores over ten iterations. Colors indicate min/max values and IndDom clusters.

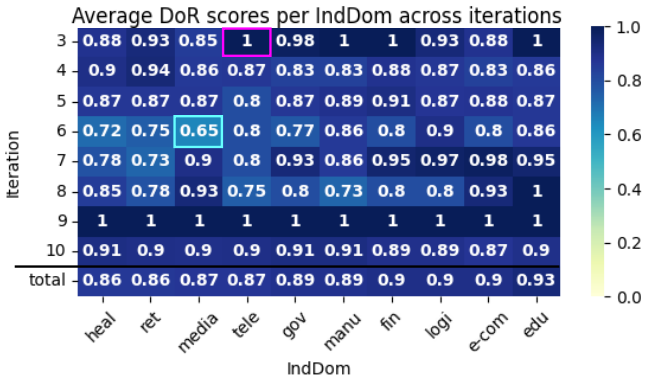

Figure 5: Heatmap of average DoR scores over ten iterations. Colors indicate min/max values.

the average similarity scores of each IndDom in each iteration as a heatmap. The last row (total) represents the overall average of the scores for one IndDom across all iterations. The IndDoms are ordered ascending by their overall average similarity score. It ranges between 0.55 (blue marker) and 0.96 (pink marker) across all iterations. Moreover, despite the tenth iteration with a low range of 0.14, the average similarity score ranges from 0.27 to 0.36 across the iterations, with a total average range of 0.34. These results indicate that in all but the last iteration, ChatGPT generated both highly repetitive and diverse SSyRSs. Moreover, the average similarity scores reveal three IndDom clusters, indicated by the green, yellow, and red areas: IndDoms with often diverse SSyRSs (green); IndDoms with SSyRSs that are balanced between diverse and similar (yellow); IndDoms with often similar SSyRSs (red).

Average DoR: The average DoR score shows inconsistent behavior. As seen in figure 3, it ranges from 0.79 in the sixth iteration to 0.95 in the third. The overall DoR average is 0.88, indicating a high confidence of ChatGPT in interpreting the SSyRSs it generated as very realistic. Even with the size growth of both P_{Gen} and P_{DoR}, reflecting more sophisticated generation and assessment instructions, no immediate effects can be identified. These observations are in line with the DoR scores within the IndDoms. Figure 5 depicts a heatmap that shows the average DoR scores of each IndDom from the third to the tenth iteration. Its structure corresponds to the heatmap in figure 4; the IndDoms are sorted in ascending order by the overall average DoR score. The score ranges from 0.65 (blue marker) to 1 (pink marker). However, lower values are outliers; the range of the overall average DoR score is only 0.07. Healthcare represents the IndDom with the least realistic SSyRSs (0.86), Education the IndDom with the most realistic SSyRSs (0.93). This shows that ChatGPT considers the SSyRS to be, in general, very realistic.

However, two things must be noted. First, high and stable DoR scores do not necessarily mean realistic SSyRSs, as they can also indicate too optimistic or less critical assessments compared to lower or more unstable DoR scores. This was, for instance, the case in the ninth iteration, which showed perfect DoR scores for all IndDoms, while we disagreed on these high scores based on subjective assessments. Second, as explained before, P_{DoR} yielded both quantitative and qualitative results. Even though trends and consistent behavior cannot be identified on a quantitative level, the qualitative explanations were indeed very important and represented the most important source for prompt refinements. Based on our subjective assessments, these also led to better results in later iterations. These effects, however, are not reflected by the numbers. To give an example, the DoR prompts in the sixth iteration often showed critique for the resource constraints, such as for the first media SSyRS of this iteration, which had a DoR score of 0.7:

```
Issue: The development budget of $2.5 million and a six-month
timeline for an MVP may be insufficient given the complexity of
integrating licensing systems with CDNs, ERPs, and third-party
analytics tools, while also meeting performance and compliance
requirements. Extending the timeline to 9-12 months would be more
realistic for such a domain-specific system.
```

These observations led to the following modification of P_{Gen}:

```
When reasoning through each section of the template step by step,
prioritize slightly conservative estimates of resources (e.g.,
time-frame, budget, team size) to ensure they are realistic and
sufficient. Assume that unforeseen challenges may arise during
development or implementation and allocate resources accordingly.
```

We terminated SSyRS-Gen after ten iterations for three reasons. We indeed observed positive effects by our prompt refinements of P_{Gen} and P_{DoR}, but their effects appeared rather random in later iterations. This also caused the size reduction of P_{Gen} from the eighth iteration onward, as we hoped to get more comprehensible results again by removing instructions from P_{Gen}. Secondly, we yielded a large dataset of 300 SSyRSs that can be used for further analysis. Lastly, the SSyRSs from the tenth iteration showed the best results for the DoR scores, similarity scores, and our subjective assessments. Thus, we considered these SSyRSs as suitable for the expert evaluation. The final dataset has an average size of 715 words, an average DoR score of 0.9, and an average similarity score of 0.66. Due to space reasons, we do not provide an example, but refer to the GitHub repository, where we share all generated SSyRSs[1].

[1]https://github.com/anonymous-science-data/Synthetic-System-Requirement-Specification-SSyRS-Repository

PREPRINT

5 Evaluation: Questionnaire Study

We conducted a questionnaire study to evaluate the SSyRSs from the final iteration. To this end, we defined the central question to be answered by the evaluation: "Are SSyRSs perceived as realistic by software experts?".

5.1 Evaluation Setup

5.1.1 Questionnaire Design. We implemented the questionnaire with the SoSci Survey platform [20]. From the 30 SSyRSs, we randomly selected one SSyRS for each IndDom and designed one questionnaire for each of these SSyRSs. At the beginning of the questionnaire, participants specified their industry background. Depending on the answer, participants were assigned only the questionnaire of the according IndDom. If participants chose multiple IndDom selections, the questionnaire was randomly assigned among these IndDoms, and after completing the questionnaire, they had the option of being assigned another questionnaire from their selected IndDoms or ending the survey. In case of no match with the industry background, the questionnaire was ended. Each questionnaire implemented the four SSyRS categories. Participants were asked to evaluate these categories individually, using two answer options: "realistic" and "unrealistic". Participants were explicitly informed that these options were meant to reflect tendencies rather than absolute correctness. Each assessment included an optional free text field that could be used for comments on the rating. In the end, participants gave an overall realism assessment on a five-point Likert scale, ranging from *very artificial* to *very realistic*.

5.1.2 Participant Selection. Participants were recruited using purposive sampling and snowball sampling. The questionnaire was advertised via personal contacts, LinkedIn, and the mailing list of the "Softwaretechnik" special interest group of the German Informatics Society (GI). We explicitly addressed individuals with professional experience in software engineering. Participation was voluntary and required informed consent, which outlined key aspects of the study, including data protection measures, the purpose of data collection, the specific data gathered, and how the data would be used. Participants were informed that their responses will be anonymized and used solely for research purposes. A contact e-mail was provided for any questions or concerns regarding data usage. No incentives were offered. To ensure valid responses despite the open distribution of the questionnaire link, we included filter questions at the start, which filtered out participants without professional software experience or without confidence in assessing a SSyRS in at least one of the IndDoms. We also asked about their experience in years and whether they identified as technical experts, domain experts, or both. Table 1 summarizes the demographics of the 83 participants. The majority of experts had over 7 years of experience, and approximately 58% identified as having domain expertise.

5.1.3 Implementation. A pretest was conducted with five participants, each of whom freely selected one of the ten available SSyRSs. The participants were personal contacts from our network. As a result, the pretest covered four different IndDoms, with two participants selecting the same domain. We measured average completion time and collected qualitative feedback to improve the questionnaire's clarity and usability. Based on the feedback, we added brief guidelines clarifying that binary judgments should reflect tendencies rather than absolute correctness. Participants were also instructed to evaluate only the presented content. Critiquing missing information was considered not meaningful, as the scope and structure of each SSyRS was predefined by the SSyRS-template. The final study was conducted between March 26 and June 15, 2025.

Table 1: Participant Demographics (N=83)

Category	Group	Percentage
Years of Experience	<1 year	8.4%
	1–3 years	16.9%
	4–6 years	13.3%
	7–10 years	50.6%
	>10 years	10.8%
Expert Type	Technical	42.2%
	Domain	19.3%
	Both	38.6%

5.1.4 Data Analysis. We exported all complete and incomplete datasets from SoSci Survey without any modification. Numeric codes used by SoSci Survey (e.g., for Likert scale responses) were mapped back to their corresponding textual labels to facilitate interpretation. The data was parsed and analyzed using the Python library `pandas` and `matplotlib.pyplot` for visualization.

5.2 Evaluation Results

In total, 87 questionnaires were submitted, of which 70 were completely answered. Four participants submitted two questionnaires. A total of 227 comments were collected. All datasets were included in the analysis, as each SSyRS section was evaluated independently, and partial responses did not introduce analytical bias. Figure 6 presents the overall rating. Combined, 61.4% of experts rated their SSyRS as either somewhat or even very realistic. In comparison, only 14,3% gave a rating of either somewhat or very artificial.

To complement the quantitative analysis and put the estimations into context, we examined the optional justification comments provided for sections marked as "unrealistic". In these responses, four recurring groups of issues were identified: 1) Oversimplification and generic phrasing indicated by comments like "Look like textbook examples"; 2) Lack of detail and ambiguity described as "That is very vague. [...] more information [...] should be defined."; 3) Logical incoherence between requirements like a misapplication of EU standards to non-EU settings; 4) Overly ambitious requirements, such as "15.000 users for an initial round seems too ambitious".

Together, the data analysis offers a nuanced answer to the question proposed for this evaluation study. On the surface, the answer appears positive, as over 60% of experts gave an overall assessment of either somewhat or even very realistic. However, deeper analysis of expert comments reveals that this perception may not accurately reflect the DoR of presented SSyRSs. Experts who conducted an in-depth analysis, as indicated by their comments, frequently identified substantial flaws, especially for the sections of *Resource*

PREPRINT

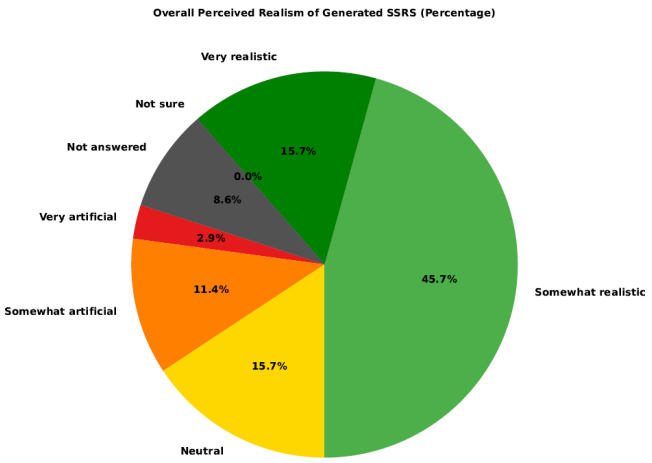

Figure 6: Proportion of participant responses for the overall realism rating of the SSyRSs.

Constraints, *User Base Characteristics*, and Non-Functional Requirements like *Scalability*, *Reliability*, and *Performance*. These findings suggest that while LLM-generated SSyRSs are presented in a confident and plausible tone, they contain important realism issues that are only uncovered through detailed expert evaluation. This is a particularly critical finding, as it demonstrates that even experienced software experts can be misled by the interplay of wrong data and overconfidence in LLM-generated content, unless they engage in close, critical examination of the provided information.

6 Discussion

The results of our study provide differentiated insights into using ChatGPT for generating realistic synthetic system requirement specifications (SSyRS). In the following, we discuss these results and give answers to the research questions (RQ1–RQ4).

6.0.1 RQ1 & RQ2: Quality properties and metrics. We defined three quality properties that SSyRSs should embody to effectively be used in SE research: *Realistic*, *Comparable*, and *Diverse*. These properties describe different but complementary quality aspects. Realistic describes how plausible a SSyRS resembles a real SyRS. Comparable ensures that a SSyRS is consistent and sufficiently structured to enable comparisons with other SSyRSs. Diverse refers to the ability that SSyRSs can model different yet closely related SyRSs. Together, these properties provide the foundation for assessing and improving SSyRSs in a systematic generation approach, such as SSyRS-Gen, and enable their use as a substitute for real SyRSs.

For each quality property, we defined a metric and calculated it for SSyRSs in an automated fashion. Consequently, no domain experts are involved in these assessments. For *Realistic*, we defined the Degree of Realism (DoR) metric. It combined both a quantitative measure in the form of the DoR score $\in [0, 1]$, and qualitative reasoning for each finding, which caused a decrease in the DoR score. While the DoR score was designed to offer an objective approximation of realism, our observations from the case study showed that they were highly inaccurate and often did not match our subjective assessments of the realism of the SSyRSs. However, the accompanying qualitative explanations did contain important and highly relevant information about weaknesses in the SSyRSs. These often corresponded to our subjective assessments and also helped to improve the quality of the SSyRSs based on subjective perception through targeted prompt refinements. The following key insight concludes these findings:

> Quantitative self-assessments of subjective properties in NL artifacts are unreliable. Qualitative assessments are an important source for identifying weaknesses in complex NL artifacts.

For *Comparable*, we implemented the *Completeness* metric. It yielded binary results by comparing whether the structure of a SSyRS conforms to the pre-defined template with which it was generated. The metric performed robustly with perfect accuracy and consistency. For *Diverse*, we measured pairwise semantic similarity scores between SSyRSs within an IndDom using SBERT. These scores enabled the quantification of the degree of deviation between SSyRSs with a close semantic relationship due to the same IndDom. While the direct influence of the prompt refinements on the semantic similarity of SSyRSs could not clearly be identified, we observed that the distribution of similarity scores aligned well with manual reviews, ranging from nearly similar instances to more diverse ones. The scores also allowed us to identify IndDom-specific trends, as certain IndDoms showed clear tendencies towards more similar, and others towards more diverse SSyRSs. Lastly, these measurements helped us to optimize the prompts in our prompt refinements in a targeted manner. This is concluded by the following key insight:

> Structural validation based on a pre-defined template is a highly effective method for quality assurance of NL artifacts. Semantic similarity is a reliable metric for diversity measurements and is of high value to identify trends in domain-related NL artifacts.

6.0.2 RQ3: Iterative prompt refinements. Our SSyRS-Gen process was centered around iterative prompt refinements, aiming to iteratively improve the SSyRSs. These iterative refinements were primarily guided by qualitative analyses, particularly the qualitative explanations yielded by the DoR metric. The analyses enabled us to identify structural and semantic deficiencies in the SSyRSs, such as contradictory requirements caused by a misalignment between resource constraints combined with other requirements, and incrementally refine the prompts to correct the deficiencies. The iterative refinement approach also helped us in sharpening the self-assessment prompt P_{DoR}, for instance, by changing the initial boolean assessment to a relational assessment in the interval $[0, 1]$.

In terms of size measures, the iterative prompt refinements had clear effects. Throughout our case study, SSyRS showed a size growth of 47% on average, which reflects more detailed and sophisticated artifacts in later iterations compared to earlier ones. In terms of the quality measures, the effects of our iterative prompt refinements are not clear. They did not affect the completeness assessments, as they were flawless from the very beginning. Positive effects on the diversity of SSyRSs through the similarity scores could be observed overall. However, other factors, such as the probabilistic nature of LLMs, cannot be ruled out as the cause. In the quantitative

DRAFT

DoR scores, positive effects of the refinements are not reflected. Thus, overall quality enhancements based on iterative prompt refinements are not reflected by our quantitative assessments. However, based on the subjective assessment of all researchers involved, SSyRSs improved over the course. This becomes apparent when analyzing SSyRSs for overly generic language, such as "the system should be reliable," which lacks concrete, domain-specific detail. Such statements occurred more frequently in the SSyRSs of earlier iterations than in later ones. The qualitative explanations of findings in the DoR self-assessments also reflect the subjective enhancements of SSyRSs. Later versions of the self-assessment prompts showed precise and less superficial explanations. While we cannot share the findings in detail in the paper due to space reasons, they can be comprehended in our artifacts [1].

Lastly, iterative prompt refinements also show fragile characteristics. Since the input prompts are characterized by instructions that are developed systematically and structurally, but are still qualitative in nature, unforeseen side effects cannot be ruled out. This was the case in the ninth iteration of our case study, in which the DoR self-assessment prompt suddenly classified all SSyRSs as 100% realistic. Although we were unable to trace the incorrect assessments entirely back to errors in the prompts, a (necessary) revision nevertheless led to good results in the following iteration. We conclude these findings by the following key insight:

> Iterative refinements of input prompts can lead to improvements in complex NL documents. Quantified metrics may not reflect this progress, making subjective evaluations through in-depth analysis of complex NL artifacts indispensable.

6.0.3 RQ4: Subjective realism of SSyRSs. To understand the perception of seemingly realistic SSyRSs based on the DoR self-assessments, we conducted an expert questionnaire study. It revealed that on a higher level, expert and LLM assessments suggest broadly similar realism judgments. 61.4% of the participants rated their SSyRSs as realistic. In comparison, ChatGPT's self-assessments were significantly more optimistic, with an average realism score of 0.90 in the final iteration and 0.88 overall. However, this alignment is only superficial. A qualitative examination of expert feedback, especially for SSyRSs rated as "somewhat realistic", revealed numerous realism flaws that challenge those positive classifications, raising the question of whether these SSyRSs should have been rated as realistic at all.

We identify three key factors contributing to this discrepancy. First, the confident and structured presentation of LLM-generated text likely creates an illusion of realism, particularly in NL formats. Since LLMs are trained primarily on public internet data, often lacking the nuanced detail found in expert-level documentation, they tend to generalize or hallucinate plausible-sounding content that may not withstand domain scrutiny. Second, participants may have been influenced by their attitudes toward AI. Research on AI appreciation and aversion shows that trust in AI is shaped by perceived capability, personalization, and social influence [19]. Knowing upfront that the SSyRSs were AI-generated could have triggered both positive bias (novelty or trust in automation) and negative skepticism (distrust or critical framing), depending on individual predispositions. Third, the depth of analysis varied. Some experts engaged deeply and uncovered specific flaws, while others may have relied on surface-level heuristics. Taken together, while LLM and expert assessments seem to agree quantitatively, a closer look reveals meaningful gaps. This emphasizes the need for careful interpretation of both human and LLM-based realism assessments. The last key insight concludes this discussion:

> LLM-generated (complex) NL artifacts are unreliable and must be thoroughly reviewed by experts, as even experts can be misled by seemingly plausible formulations and overconfidence.

6.1 Threats to Validity

6.1.1 Construct Validity. Our study is obviously based on self-assessments generated by ChatGPT to evaluate the defined quality properties of SSyRSs. Consequently, these assessments may not reflect the quality properties as perceived by a human. While the pure structural completeness metric was very reliable, this was not the case for the DoR assessments. Furthermore, ChatGPT's overestimation of its own abilities and lack of in-depth domain-specific reasoning may lead to biased assessments. However, since the focus of this study was to examine ChatGPT's performance for our use case, this threat was evident. Based on this, the quality assessments of our SSyRSs cannot be considered universally correct. The expert study was able to mitigate this threat, but not completely avoid it.

Moreover, we used a synthetic and simplified template for SSyRS generation. This might not capture the full complexity of SyRSs and can be an influencing factor on how realistic the SSyRSs are perceived. In designing the template, we took great care to ensure that all but the most technical parts, which are difficult to add plausibly when there is no actual system given, such as existing interface definitions, are included in the template in accordance with the standard [16]. We therefore do not consider the simplifications made to be so fundamental that they significantly distort the quality assessments. However, this possibility cannot be ruled out.

6.1.2 Internal Validity. We observed improvements in the semantic similarities and in our subjective assessments throughout the iterations. While the prompt refinements can have led to these improvements, as discussed earlier, the probabilistic nature of LLMs cannot be ruled out as a possible cause for the improvements. Furthermore, we implemented our prompt engineering approach subjectively, making it even more difficult to attribute observed changes in the quality of SSyRSs to specific prompt changes. This complicates causal interpretation. We deliberately decided against a systematic approach due to the sheer complexity of the design space. Not only are there numerous prompt patterns to consider, but the effect of each pattern can vary significantly depending on its specific wording and contextual integration, for example, depending on which expert role is used in a persona prompt. However, due to this risk, we deliberately avoided generalized causal relationships between the prompt changes and the effects on the results and pursued an exploratory approach throughout the study.

6.1.3 External Validity. A key limitation of the expert study relates to participant sampling and qualification verification. The study employed purposive sampling with additional snowball recruitment, deliberately targeting individuals with relevant professional

PREPRINT

backgrounds. While this approach aimed to attract qualified experts, it does not allow for full verification of participant expertise. Although several screening questions were used, including filters for professional software experience, self-identification as a domain and/or technical expert, years of experience, and domain-specific confidence, these relied on self-reporting and could not be independently verified. Consequently, while the participant pool likely included mostly qualified experts, the possibility remains that some respondents lacked the depth of expertise required for a rigorous realism evaluation. This limitation introduces some uncertainty regarding the consistency and reliability of the expert judgments, though the study design aimed to minimize this risk through targeted recruitment and domain and expertise filtering.

Next, the expert study involved a single SSyRS per domain. This small sample size limits statistical power and reduces the generalizability of the findings. Therefore, we did not compare the results of the individual IndDoms with each other and considered them as a total sample of the 30 final SSyRSs. Since these are semantically different, it is nevertheless likely that the choice of IndDom may have had an influence on the expert assessment. This design decision allowed us to examine a larger overall sample, as we were able to identify multiple experts through the large number of IndDoms. In return, we had to accept the disadvantages of limited statistical power and reduced generalizability. However, we consider this a lesser evil, considering that the alternative would have been a smaller overall sample for one IndDom, which would have prevented the results from being generalized as well.

Lastly, the design of the case study poses another limitation, as it was limited to ten industry domains that we defined upfront. Consequently, the results of our study cannot be transferred to industry domains that were not included in the case study.

7 Related Work

Cosler et al. [7] developed the nl2spec framework, capable of translating unstructured NL requirements into formal specifications in temporal logics. Zhang et al. [40] propose an AI-aided Software Development approach. Among other things, it takes user requirements as input and generates use cases from them. Iqbal et al. [17] developed a framework for automatically generating software requirements for new mobile applications by mining app store content and competitor feedback. Yeow et al. [39] developed an LLM-based approach using GPT-3.5 to generate survey and interview questions for requirements elicitation across various industry domains. They assess the linguistic quality and contextual relevance of the LLM-generated questions using metrics such as Flesch Reading Ease and expert evaluation. Moreover, Peer et al. [27] developed the "Natural Language Processing for Requirement Forecasting" (NLP4ReF) framework, which utilizes a natural language toolkit and ChatGPT to classify requirements by functional and non-functional requirements as well as their respective system classes, and to generate overlooked requirements from an existing requirements set.

These approaches share a common goal with our study: generating realistic RE artifacts. However, they all differ from our approach in terms of their input, output, or the methods used to generate the data. Iqbal et al. [17] address the same problem as SSyRS-Gen: scenarios where real-world requirement specifications or expert input are unavailable, and to generate RE artifacts without relying on existing user data. However, the authors do not use an LLM but an ML-pipeline based on NLP and traditional ML techniques to generate data. SSyRS-Gen uses a black-box LLM instead. Moreover, their approach generates software requirements only, while SSyRS-Gen generates excerpts of SyRSs that add context to the requirements. Cosler et al. [7] use existing requirements in NL as input, while SSyRS-Gen uses replaceable prompt templates, thus having no dependencies on available data at all. Moreover, the generated data of nl2spec are formal specifications in temporal logics, while SSyRS-Gen generates full specifications in NL. Zhan et al. [40] generate use cases while SSyRS-Gen generates SSyRS. Moreover, AISD requires user requirements as input to generate use cases. SSyRS-Gen does not depend on any historic data. Yeow et al. [39] focus on facilitating the elicitation phase for a predefined system by automatically generating survey and interview questions for requirements elicitation. In contrast, SSyRS-Gen generates requirements, and it does that for a novel, synthetic system.

Lastly, our work is related to the PROMISE dataset, which is considered a gold standard in this field [32]. The data set offers a wide range of functional and non-functional requirements and can therefore be compared with the functional and non-functional requirements of our SSyRSs. However, each SSyRS defines additional and important contexts. This additional context has a significant effect on the possible uses of these requirements, as the requirements can be interpreted and used depending on the context.

8 Conclusion

This study examines whether ChatGPT can be used to generate realistic synthetic system requirement specifications (SSyRSs). It introduces three quality properties to be assured when generating SSyRSs and metrics to calculate them without domain expert involvement. The underlying generation approach (SSyRS-Gen) follows an iterative prompt refinement approach based on diagnostic data from earlier iterations. We implemented and executed SSyRS-Gen in a case study, generating 300 SSyRSs for ten different industry domains (IndDoms). We shared all SSyRSs in a public GitHub repository. Moreover, all prompts, self-assessments, and their results are publicly available [1]. An expert questionnaire study revealed that over 60% of the participants perceived a SSyRSs sample as rather realistic, in line with ChatGPT's self-assessments. However, in-depth analyses of the SSyRSs raised serious concerns about these assessments. Our results show the dangerous interplay of human perception, LLM overconfidence, and the plausible formulation that LLMs tend to generate. We conclude that the temptation to use ChatGPT for generating complex natural language artifacts, especially in expert environments, should be resisted. And if it is used, in-depth expert reviews of the data are inevitable.

For future work, the functional and non-functional requirements can be extracted from our SSyRSs and investigated to serve as an alternative dataset to PROMISE. Moreover, we plan extensive follow-up expert evaluations that allow for more statistical power, for instance, by comparing expert perception of the SSyRSs yielded from the first iteration with those from the tenth iteration.

PREPRINT

References

[1] Anonymous. 2025. *Anonymous*. doi:10.5281/zenodo.16146454

[2] Len Bass, Qinghua Lu, Ingo Weber, and Liming Zhu. 2025. *Engineering AI Systems: Architecture and DevOps Essentials*. Addison-Wesley, New Jersey.

[3] Pronaya Bhattacharya, Vivek Kumar Prasad, Ashwin Verma, Deepak Gupta, Assadaporn Sapsomboon, Wattana Viriyasitavat, and Gaurav Dhiman. 2024. Demystifying ChatGPT: An In-depth Survey of OpenAI's Robust Large Language Models. *Archives of Computational Methods in Engineering* 31, 8 (2024), 4557–4600. doi:10.1007/s11831-024-10115-5

[4] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin (Eds.), Vol. 33. Curran Associates, Inc, 1877–1901. https://proceedings.neurips.cc/paper_files/paper/2020/file/1457c0d6bfcb4967418bfb8ac142f64a-Paper.pdf

[5] Dhivya Chandrasekaran and Vijay Mago. 2021. Evolution of Semantic Similarity—A Survey. *ACM Comput. Surv.* 54, 2, Article 41 (Feb. 2021), 37 pages. doi:10.1145/3440755

[6] Qinyuan Cheng, Tianxiang Sun, Xiangyang Liu, Wenwei Zhang, Zhangyue Yin, Shimin Li, Linyang Li, Zhengfu He, Kai Chen, and Xipeng Qiu. 2024. Can AI Assistants Know What They Don't Know? arXiv:2401.13275 [cs.CL] https://arxiv.org/abs/2401.13275

[7] Matthias Cosler, Christopher Hahn, Daniel Mendoza, Frederik Schmitt, and Caroline Trippel. 2023. nl2spec: Interactively Translating Unstructured Natural Language to Temporal Logics with Large Language Models. In *Computer Aided Verification*, Constantin Enea and Akash Lal (Eds.). Springer Nature Switzerland, Cham, 383–396.

[8] Brian Dean. 2024. ChatGPT User and Growth Stats (2024). https://explodingtopics.com/blog/chatgpt-users. https://explodingtopics.com/blog/chatgpt-users Accessed: 2025-07-17.

[9] Emelie Engström, Margaret-Anne Storey, Per Runeson, Martin Höst, and Maria Teresa Baldassarre. 2020. How software engineering research aligns with design science: a review. *Empirical Software Engineering* 25, 4 (2020), 2630–2660. doi:10.1007/s10664-020-09818-7

[10] Alessio Ferrari, Felice Dell'Orletta, Andrea Esuli, Vincenzo Gervasi, and Stefania Gnesi. 2017. Natural Language Requirements Processing: A 4D Vision. *IEEE Software* 34, 6 (2017), 28–35. doi:10.1109/MS.2017.4121207

[11] A. Ferrari, G. O. Spagnolo, and S. Gnesi. 2017. PURE: A Dataset of Public Requirements Documents. In *2017 IEEE 25th International Requirements Engineering Conference (RE)*. IEEE, Lisbon, Portugal, 502–505. doi:10.1109/RE.2017.29

[12] Jiawei Gu, Xuhui Jiang, Zhichao Shi, Hexiang Tan, Xuehao Zhai, Chengjin Xu, Wei Li, Yinghan Shen, Shengjie Ma, Honghao Liu, Saizhuo Wang, Kun Zhang, Yuanzhuo Wang, Wen Gao, Lionel Ni, and Jian Guo. 2025. A Survey on LLM-as-a-Judge. arXiv:2411.15594 [cs.CL] https://arxiv.org/abs/2411.15594

[13] Havva Gulay Gurbuz and Bedir Tekinerdogan. 2018. Model-based testing for software safety: a systematic mapping study. *Software Quality Journal* 26, 4 (2018), 1327–1372. doi:10.1007/s11219-017-9386-2

[14] Mengting Hu, Zhen Zhang, Shiwan Zhao, Minlie Huang, and Bingzhe Wu. 2023. Uncertainty in Natural Language Processing: Sources, Quantification, and Applications. arXiv:2306.04459 [cs.CL] https://arxiv.org/abs/2306.04459

[15] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, and Ting Liu. 2025. A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions. *ACM Trans. Inf. Syst.* 43, 2 (2025). doi:10.1145/3703155

[16] International Organization for Standardization, International Electrotechnical Commission, and Institute of Electrical and Electronics Engineers. 2018. *ISO/IEC/IEEE 29148:2018 — Systems and software engineering — Life cycle processes — Requirements engineering*. Technical Report ISO/IEC/IEEE 29148:2018. ISO/IEC/IEEE.

[17] Tahira Iqbal, Norbert Seyff, and Daniel Mendez. 2019. Generating Requirements Out of Thin Air: Towards Automated Feature Identification for New Apps. In *2019 IEEE 27th International Requirements Engineering Conference Workshops (REW)*. 193–199. doi:10.1109/REW.2019.00040

[18] Mohamad Kassab, Colin Neill, and Phillip Laplante. 2014. State of practice in requirements engineering: contemporary data. *Innovations in Systems and Software Engineering* 10, 4 (2014), 235–241. doi:10.1007/s11334-014-0232-4

[19] Artur Klingbeil, Cassandra Grützner, and Philipp Schreck. 2024. Trust and reliance on AI — An experimental study on the extent and costs of overreliance on AI. *Computers in Human Behavior* 160 (2024), 108352. doi:10.1016/j.chb.2024.108352

[20] D. J. Leiner. 2025. SoSci Survey (Version 3.7.06) [Computer software]. Available at https://www.soscisurvey.de.

[21] Xun Liang, Shichao Song, Zifan Zheng, Hanyu Wang, Qingchen Yu, Xunkai Li, Rong-Hua Li, Yi Wang, Zhonghao Wang, Feiyu Xiong, and Zhiyu Li. 2024. Internal Consistency and Self-Feedback in Large Language Models: A Survey. arXiv:2407.14507 [cs.CL] https://arxiv.org/abs/2407.14507

[22] Mich Luisa, Franch Mariangela, and Novi Inverardi Pierluigi. 2004. Market research for requirements analysis using linguistic tools. *Requirements Engineering* 9, 1 (2004), 40–56. doi:10.1007/s00766-003-0179-8

[23] Potsawee Manakul, Adian Liusie, and Mark J. F. Gales. 2023. SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models. arXiv:2303.08896 [cs.CL] https://arxiv.org/abs/2303.08896

[24] Ali Mehraj, Zheying Zhang, and Kari Systä. 2024. A Tertiary Study on AI for Requirements Engineering. In *Requirements Engineering: Foundation for Software Quality*, Daniel Mendez and Ana Moreira (Eds.). Springer Nature Switzerland, Cham, 159–177.

[25] Allen Nie, Yash Chandak, Miroslav Suzara, Malika Ali, Juliette Woodrow, Matt Peng, Mehran Sahami, Emma Brunskill, and Chris Piech. 2024. The GPT Surprise: Offering Large Language Model Chat in a Massive Coding Class Reduced Engagement but Increased Adopters Exam Performances. arXiv:2407.09975 [cs.CY] https://arxiv.org/abs/2407.09975

[26] Janis Osis and Armands Šlihte. 2010. Transforming Textual Use Cases to a Computation Independent Model. In *Proceedings of the 2nd International Workshop on Model-Driven Architecture and Modeling Theory-Driven Development*, Janis Osis and Oksana Nikiforova (Eds.). SciTePress, Portugal, 33–42.

[27] Jordan Peer, Yaniv Mordecai, and Yoram Reich. 2024. NLP4ReF: Requirements Classification and Forecasting: From Model-Based Design to Large Language Models. In *2024 IEEE Aerospace Conference*. 1–16. doi:10.1109/AERO58975.2024.10521022

[28] Ken Peffers, Tuure Tuunanen, Marcus A. Rothenberger, and Samir Chatterjee. 2007. A Design Science Research Methodology for Information Systems Research. *Journal of Management Information Systems* 24, 3 (2007), 45–77. doi:10.2753/MIS0742-1222240302

[29] Aske Plaat, Annie Wong, Suzan Verberne, Joost Broekens, Niki van Stein, and Thomas Back. 2024. Reasoning with Large Language Models, a Survey. arXiv:2407.11511 [cs.AI] https://arxiv.org/abs/2407.11511

[30] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics. https://arxiv.org/abs/1908.10084

[31] Dylan Sam, Marc Finzi, and J. Zico Kolter. 2025. Predicting the Performance of Black-box LLMs through Self-Queries. arXiv:2501.01558 [cs.LG] https://arxiv.org/abs/2501.01558

[32] J. Sayyad Shirabad and T. J. Menzies. 2005. The PROMISE Repository of Software Engineering Databases. Technical report, School of Information Technology and Engineering, University of Ottawa. http://promise.site.uottawa.ca/SERepository Accessed 19 July 2025.

[33] Kayla Schroeder and Zach Wood-Doughty. 2025. Can You Trust LLM Judgments? Reliability of LLM-as-a-Judge. arXiv:2412.12509 [cs.CL] https://arxiv.org/abs/2412.12509

[34] Sander Schulhoff, Michael Ilie, Nishant Balepur, Konstantine Kahadze, Amanda Liu, Chenglei Si, Yinheng Li, Aayush Gupta, HyoJung Han, Sevien Schulhoff, Pranav Sandeep Dulepet, Saurav Vidyadhara, Dayeon Ki, Sweta Agrawal, Chau Pham, Gerson Kroiz, Feileen Li, Hudson Tao, Ashay Srivastava, Hevander Da Costa, Saloni Gupta, Megan L. Rogers, Inna Goncearenco, Giuseppe Sarli, Igor Galynker, Denis Peskoff, Marine Carpuat, Jules White, Shyamal Anadkat, Alexander Hoyle, and Philip Resnik. 2025. The Prompt Report: A Systematic Survey of Prompt Engineering Techniques. arXiv:2406.06608 [cs.CL] https://arxiv.org/abs/2406.06608

[35] Margaret-Anne Storey, Emelie Engstrom, Martin Host, Per Runeson, and Elizabeth Bjarnason. 2017. Using a Visual Abstract as a Lens for Communicating and Promoting Design Science Research in Software Engineering. In *2017 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*. IEEE, 181–186. doi:10.1109/ESEM.2017.28

[36] Fengfei Sun, Ningke Li, Kailong Wang, and Lorenz Goette. 2025. Large Language Models are overconfident and amplify human bias. arXiv:2505.02151 [cs.SE] https://arxiv.org/abs/2505.02151

[37] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, brian ichter, Fei Xia, Ed Chi, Quoc V Le, and Denny Zhou. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh (Eds.), Vol. 35. Curran Associates, Inc., 24824–24837. https://proceedings.neurips.cc/paper_files/paper/2022/file/9d5609613524ecf4f15af0f7b31abca4-Paper-Conference.pdf

[38] Jules White, Sam Hays, Quchen Fu, Jesse Spencer-Smith, and Douglas C. Schmidt. 2023. ChatGPT Prompt Patterns for Improving Code Quality, Refactoring, Requirements Elicitation, and Software Design. arXiv:2303.07839 [cs.SE] https://arxiv.org/abs/2303.07839

PREPRINT

[39] Jie Sh'ng Yeow, Muhammad Ehsan Rana, and Nur Amira Abdul Majid. 2024. An Automated Model of Software Requirement Engineering Using GPT-3.5. In *2024 ASU International Conference in Emerging Technologies for Sustainability and Intelligent Systems (ICETSIS)*. 1746–1755. doi:10.1109/ICETSIS61505.2024.10459458

[40] Simiao Zhang, Jiaping Wang, Guoliang Dong, Jun Sun, Yueling Zhang, and Geguang Pu. 2024. Experimenting a New Programming Practice with LLMs. arXiv:2401.01062 [cs.SE] https://arxiv.org/abs/2401.01062

PREPRINT